MACHINE LEARNING LABORATORY - OBSERVATION

1. Experiment No.: 4	**Date:** _29/10/26

2. Experiment Question

Build an Artificial Neural Network by implementing the Backpropagation algorithm and test the same using appropriate data sets.

3. Aim

To implement a multilayer feed-forward Artificial Neural Network trained with the Backpropagation algorithm, and to test it on the XOR problem and the Iris data set.

4. Concept

An Artificial Neural Network (ANN) is a network of simple processing units (neurons) arranged in layers. Each neuron computes a weighted sum of its inputs and passes it through an activation function. A single neuron cannot solve problems that are not linearly separable (such as XOR), so a hidden layer is added.

Backpropagation trains the network by gradient descent. In the forward pass the inputs flow through the network and the output error is measured. In the backward pass the error is propagated from the output layer to the hidden layer, the contribution of each weight to the error is computed with the chain rule, and every weight is adjusted a small step against the gradient. This is repeated for many epochs until the error is small.

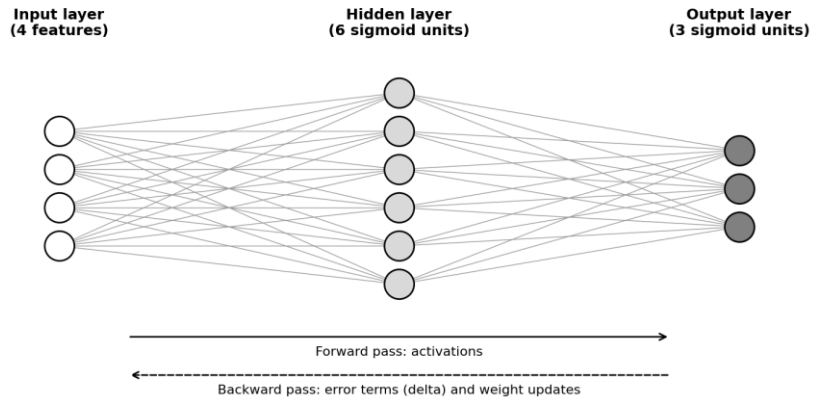


Fig. 4.1 Network architecture used for the Iris data set (4-6-3)

5. Dataset Description

Two data sets are used.

Data set	Samples	Network	Description
XOR	4	2 - 4 - 1	Inputs (x1, x2) in {0,1}. Target is 1 when exactly one input is 1. Not linearly separable.

Data set	Samples	Network	Description
Iris	150 (105 train, 45 test)	4 - 6 - 3	Sepal length, sepal width, petal length, petal width (cm). Three classes: setosa, versicolor, virginica, 50 samples each. Features are scaled to [0, 1] and classes are one-hot encoded. Split 70:30 with stratification.

x1	x2	XOR target
0	0	0
0	1	1
1	0	1
1	1	0

6. Algorithm

1. Initialise all weights with small random values and all biases with zero. Choose the learning rate (eta).
2. Forward pass: compute the hidden-layer activations, then the output-layer activations using the sigmoid function.
3. Compute the error E between the target values t and the network outputs o.
4. Output-layer error term: delta_k = (t_k - o_k) o_k (1 - o_k).
5. Hidden-layer error term: delta_h = o_h (1 - o_h) SUM[w_hk delta_k].
6. Update every weight: w_ji = w_ji + eta * delta_j * x_ji. Update the biases in the same way.
7. Repeat steps 2 to 6 for the required number of epochs, then test the trained network on unseen data.

7. Step-by-step Procedure

1. Import NumPy, scikit-learn (for loading Iris and splitting it) and Matplotlib.
2. Define the sigmoid function and its derivative.
3. Create a NeuralNetwork class with weight matrices W1 (input to hidden) and W2 (hidden to output) and methods forward(), backward() and train().
4. Train the 2-4-1 network on the XOR data for 10000 epochs with a learning rate of 1.0 and display the outputs.
5. Load Iris, scale the features, one-hot encode the labels and split into training and testing sets.
6. Train the 4-6-3 network for 5000 epochs with a learning rate of 1.0.
7. Evaluate the training and testing accuracy and plot the error against epochs.

8. Formulae

Net input and activation of a neuron

$$net_j = SUM[w_{ji} * x_i] + b_j, \qquad o_j = 1 / (1 + e^{(-net_j)})$$

Derivative of the sigmoid

$$do/dnet = o (1 - o)$$

Error (sum of squared errors)

$$E = (1/2) * SUM[(t_k - o_k)^2]$$

Error terms

$$delta_k = (t_k - o_k) * o_k * (1 - o_k) \qquad (output\ unit\ k)$$

$$delta_h = o_h * (1 - o_h) * SUM[w_{hk} * delta_k] \qquad (hidden\ unit\ h)$$

Weight update rule

$$w_{ji}\ (new) = w_{ji}\ (old) + eta * delta_j * x_{ji}$$

9. Python Program

```
"""
Experiment 4: Artificial Neural Network with the Backpropagation Algorithm
Datasets : (a) XOR problem   (b) Iris data set
Network  : input layer -> one hidden layer (sigmoid) -> output layer (sigmoid)
"""
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler


def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-z))


def sigmoid_derivative(a):          # a = sigmoid(z)
    return a * (1.0 - a)


class NeuralNetwork:
    def __init__(self, n_in, n_hidden, n_out, lr=0.5, seed=1):
        rng = np.random.default_rng(seed)
        self.W1 = rng.uniform(-1, 1, (n_in, n_hidden))
        self.b1 = np.zeros((1, n_hidden))
        self.W2 = rng.uniform(-1, 1, (n_hidden, n_out))
        self.b2 = np.zeros((1, n_out))
        self.lr = lr


    def forward(self, X):
        self.A1 = sigmoid(X @ self.W1 + self.b1)        # hidden activation
        self.A2 = sigmoid(self.A1 @ self.W2 + self.b2)  # output activation
        return self.A2


    def backward(self, X, Y):
        m = X.shape[0]
        # output-layer error term  delta_k = (t - o) * o(1 - o)
        d2 = (Y - self.A2) * sigmoid_derivative(self.A2)
        # hidden-layer error term  delta_h = o_h(1 - o_h) * sum(w_hk * delta_k)
        d1 = (d2 @ self.W2.T) * sigmoid_derivative(self.A1)
        # weight updates  w = w + eta * delta * input
        self.W2 += self.lr * self.A1.T @ d2 / m
        self.b2 += self.lr * d2.sum(axis=0, keepdims=True) / m
        self.W1 += self.lr * X.T @ d1 / m
        self.b1 += self.lr * d1.sum(axis=0, keepdims=True) / m


    def train(self, X, Y, epochs):
        losses = []
        for _ in range(epochs):
            out = self.forward(X)
            losses.append(0.5 * np.mean(np.sum((Y - out) ** 2, axis=1)))
            self.backward(X, Y)
        return losses


    def predict(self, X):
        return self.forward(X)


# ============================================================ (a) XOR
X_xor = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=float)
Y_xor = np.array([[0], [1], [1], [0]], dtype=float)


print("=" * 60)
print("(a) XOR problem  (2-4-1 network, 10000 epochs, lr = 1.0)")
print("=" * 60)
nn_xor = NeuralNetwork(2, 4, 1, lr=1.0, seed=3)
loss_xor = nn_xor.train(X_xor, Y_xor, 10000)
pred = nn_xor.predict(X_xor)
print(f"Final loss: {loss_xor[-1]:.5f}")
print(f"{'x1':>3} {'x2':>3} {'target':>7} {'output':>8} {'class':>6}")
for x, t, p in zip(X_xor, Y_xor, pred):
    print(f"{int(x[0]):>3} {int(x[1]):>3} {int(t[0]):>7} {p[0]:>8.4f} {int(p[0] > 0.5):>6}")


# ============================================================ (b) IRIS
print("\n" + "=" * 60)
print("(b) Iris data set  (4-6-3 network, 5000 epochs, lr = 1.0)")
print("=" * 60)
iris = load_iris()
X = MinMaxScaler().fit_transform(iris.data)
Y = np.eye(3)[iris.target]                          # one-hot encoding
X_tr, X_te, Y_tr, Y_te = train_test_split(X, Y, test_size=0.3, random_state=42,
                                          stratify=iris.target)
nn_iris = NeuralNetwork(4, 6, 3, lr=1.0, seed=1)
loss_iris = nn_iris.train(X_tr, Y_tr, 5000)
```

```
tr_acc = np.mean(nn_iris.predict(X_tr).argmax(1) == Y_tr.argmax(1))
te_pred = nn_iris.predict(X_te).argmax(1)
te_acc = np.mean(te_pred == Y_te.argmax(1))
print(f"Training samples: {len(X_tr)}   Testing samples: {len(X_te)}")
print(f"Final training loss : {loss_iris[-1]:.5f}")
print(f"Training accuracy   : {100*tr_acc:.2f}%")
print(f"Testing accuracy    : {100*te_acc:.2f}%")
print("\nSample predictions on the test set:")
print(f"{'Actual':<12}{'Predicted':<12}")
for a, p in list(zip(Y_te.argmax(1), te_pred))[:8]:
    print(f"{iris.target_names[a]:<12}{iris.target_names[p]:<12}")


# ============================================================ plots
fig, ax = plt.subplots(1, 2, figsize=(9, 3.4))
ax[0].plot(loss_xor, color="black"); ax[0].set_title("XOR - training loss")
ax[1].plot(loss_iris, color="black"); ax[1].set_title("Iris - training loss")
for a in ax:
    a.set_xlabel("Epoch"); a.set_ylabel("Error (E)"); a.grid(True, color="#cccccc", lw=0.5)
plt.tight_layout(); plt.savefig("figs/exp4_loss.png", dpi=200); plt.close()
```

10. Output

```
============================================================
(a) XOR problem  (2-4-1 network, 10000 epochs, lr = 1.0)
============================================================
Final loss: 0.00027
 x1  x2  target   output  class
  0   0       0   0.0203      0
  0   1       1   0.9771      1
  1   0       1   0.9770      1
  1   1       0   0.0268      0


============================================================
(b) Iris data set  (4-6-3 network, 5000 epochs, lr = 1.0)
============================================================
Training samples: 105   Testing samples: 45
Final training loss : 0.01214
Training accuracy   : 100.00%
Testing accuracy    : 91.11%


Sample predictions on the test set:
Actual      Predicted
virginica   virginica
versicolor  versicolor
virginica   versicolor
versicolor  versicolor
virginica   virginica
virginica   virginica
versicolor  versicolor
versicolor  versicolor
```

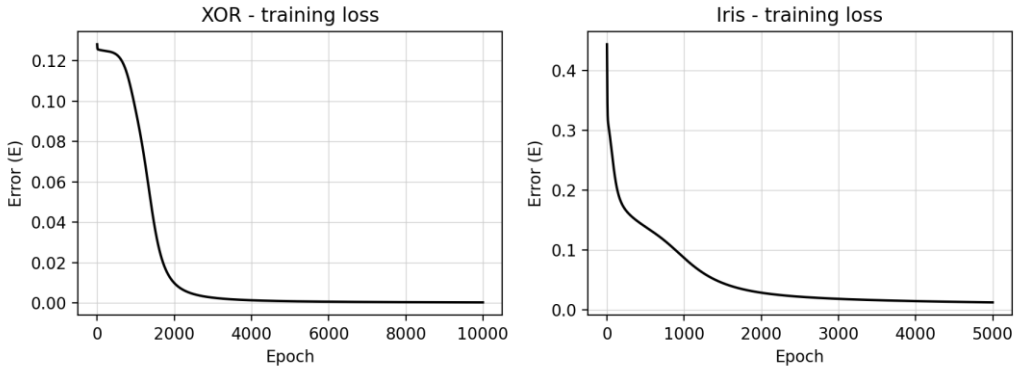


Fig. 4.2 Training error versus epochs

11. Result / Conclusion

The Backpropagation network learned the XOR function (all four outputs are within 0.03 of their targets) and classified the Iris data with 100% training accuracy and 91.11% testing accuracy. The error decreases steadily with the number of epochs, which shows that the weight updates move the network towards a minimum of the error. Hence the Backpropagation algorithm was implemented and tested successfully.